OpenCV 中的直方图比较

笔记作者

2026 年 3 月 28 日

目录

1 概述

直方图比较是计算机视觉中一种重要的图像相似性度量技术，通过比较两幅图像直方图的分布差异来评估图像的相似程度。在 OpenCV 中，直方图比较广泛应用于图像检索、对象识别、图像分类和质量控制等领域。

直方图比较的核心思想是将图像的颜色或灰度分布抽象为直方图，然后使用各种统计距离度量方法计算两个直方图之间的相似度。这种方法对图像的几何变换（如旋转、缩放）具有一定的鲁棒性，特别适用于基于内容的图像检索系统。

2 基本概念

定义 2.1 (直方图相似度)**.** *衡量两个直方图分布相似程度的量化指标。通过计算两个直方图之间的距离或相似度得分来评估图像之间的相似性。*

定义 2.2 (距离度量)**.** *用于计算两个分布之间差异的数学方法。OpenCV 提供了多种距离度量方法，包括相关性、卡方距离、相交距离、巴氏距离和 Hellinger 距离。*

定义 2.3 (归一化直方图)**.** *将直方图转换为概率分布的过程。通过除以总像素数，使每个 bin 的值表示该灰度级出现的概率：*

$$p(i) = \frac{h(i)}{MN}$$

其中 $h(i)$ 是原始直方图，$M \times N$ 是图像尺寸。

定义 2.4 (累积直方图)**.** *直方图的累积分布函数，定义为：*

$$C(k) = \sum_{i=0}^{k} p(i)$$

用于某些直方图比较方法中。

定义 2.5 (多维直方图)**.** *考虑颜色和空间信息的复杂直方图表示。对于彩色图像，可以计算各颜色通道的联合直方图，如 2D 直方图（H 和 S 通道）或 3D 直方图（R、G、B 通道）。*

注记 2.1. *直方图比较通常需要先对直方图进行归一化处理，以避免图像尺寸差异对比较结果的影响。*

3 基本原理

OpenCV 提供了多种直方图比较方法，每种方法基于不同的数学原理：

3.1 相关性比较（HISTCMP_CORREL）

基于皮尔逊相关系数，测量两个直方图之间的线性相关性：

$$d(H_1, H_2) = \frac{\sum_I (H_1(I) - \bar{H}_1)(H_2(I) - \bar{H}_2)}{\sqrt{\sum_I (H_1(I) - \bar{H}_1)^2 \sum_I (H_2(I) - \bar{H}_2)^2}} \tag{1}$$

其中 $\bar{H}_1$ 和 $\bar{H}_2$ 分别是两个直方图的均值。结果范围：$[-1, 1]$，1 表示完全正相关，-1 表示完全负相关，0 表示不相关。

3.2 卡方比较（HISTCMP_CHISQR）

基于卡方统计量，测量两个分布的差异：

$$d(H_1, H_2) = \sum_I \frac{(H_1(I) - H_2(I))^2}{H_1(I) + H_2(I)} \tag{2}$$

结果范围：$[0, \infty)$，0 表示完全匹配，值越大差异越大。

3.3 相交比较（HISTCMP_INTERSECT）

计算两个直方图在每个 bin 上的最小值之和：

$$d(H_1, H_2) = \sum_I \min(H_1(I), H_2(I)) \tag{3}$$

结果范围：$[0, 1]$，1 表示完全匹配，0 表示完全不匹配。

3.4 巴氏距离（HISTCMP_BHATTACHARYYA）

基于巴氏系数，测量两个概率分布的重叠程度：

$$d(H_1, H_2) = \sqrt{1 - \frac{1}{\sqrt{\bar{H}_1 \bar{H}_2 N^2}} \sum_I \sqrt{H_1(I) \cdot H_2(I)}} \tag{4}$$

结果范围：$[0, 1]$，0 表示完全匹配，1 表示完全不匹配。

3.5 Hellinger 距离（HISTCMP_HELLINGER）

与巴氏距离密切相关，测量两个概率分布之间的差异：

$$d(H_1, H_2) = \sqrt{1 - \sum_I \sqrt{\frac{H_1(I)}{N_1} \cdot \frac{H_2(I)}{N_2}}} \tag{5}$$

其中 N_1, N_2 分别是两个直方图的总计数。

4 解决的问题

直方图比较技术主要解决以下问题：

- **图像相似性检索**：在大型图像数据库中快速找到与查询图像相似的图像
- **对象识别与跟踪**：通过比较直方图识别和跟踪特定对象
- **图像分类**：基于颜色或纹理特征对图像进行分类
- **质量控制**：比较产品图像与标准样本，检测缺陷
- **图像拼接**：评估图像重叠区域的相似度
- **视频分析**：检测场景变化或识别关键帧

5 使用场景

5.1 电子商务图像搜索

用户上传商品图片，系统通过直方图比较找到相似商品。这种方法对商品的角度、大小变化具有一定鲁棒性。

5.2 医学图像分析

比较医学影像（如 X 光片、CT 扫描），辅助疾病诊断和治疗效果评估。通过比较病变区域与正常组织的直方图差异。

5.3 安防监控

通过颜色特征识别特定车辆或人物。在监控视频中跟踪目标的颜色特征变化。

5.4 艺术与文化

比较绘画作品的色彩分布，研究艺术风格和画家特征。分析不同时期、不同画家的色彩使用偏好。

5.5 工业检测

检测产品外观是否符合标准。通过比较产品图像与标准样本的直方图，快速识别缺陷产品。

5.6 遥感图像处理

比较不同时间段的卫星图像，监测地表变化。如森林覆盖变化、城市扩张等。

6 代码示例

6.1 C++ 示例

Listing 1: OpenCV 直方图比较 C++ 示例

```cpp
#include <opencv2/opencv.hpp>
#include <iostream>
#include <vector>

int main() {
    // 读取两幅图像
    cv::Mat img1 = cv::imread("image1.jpg");
    cv::Mat img2 = cv::imread("image2.jpg");

    if (img1.empty() || img2.empty()) {
        std::cerr << "无法读取图像文件" << std::endl;
        return -1;
    }

    // 转换为HSV颜色空间（更适合颜色直方图比较）
```

```
    cv::Mat hsv1, hsv2;
    cv::cvtColor(img1, hsv1, cv::COLOR_BGR2HSV);
    cv::cvtColor(img2, hsv2, cv::COLOR_BGR2HSV);

    // 设置直方图参数
    int h_bins = 50, s_bins = 60;
    int histSize[] = {h_bins, s_bins};

    // Hue范围: 0-179, Saturation范围: 0-255
    float h_ranges[] = {0, 180};
    float s_ranges[] = {0, 256};
    const float* ranges[] = {h_ranges, s_ranges};

    // 使用第0和第1通道
    int channels[] = {0, 1};

    // 计算直方图
    cv::Mat hist1, hist2;
    cv::calcHist(&hsv1, 1, channels, cv::Mat(), hist1, 2, histSize, ranges, true, false);
    cv::calcHist(&hsv2, 1, channels, cv::Mat(), hist2, 2, histSize, ranges, true, false);

    // 归一化直方图
    cv::normalize(hist1, hist1, 0, 1, cv::NORM_MINMAX, -1, cv::Mat());
    cv::normalize(hist2, hist2, 0, 1, cv::NORM_MINMAX, -1, cv::Mat());

    // 比较直方图
    double correlate = cv::compareHist(hist1, hist2, cv::HISTCMP_CORREL);
    double chisquare = cv::compareHist(hist1, hist2, cv::HISTCMP_CHISQR);
    double intersect = cv::compareHist(hist1, hist2, cv::HISTCMP_INTERSECT);
    double bhattacharyya = cv::compareHist(hist1, hist2, cv::HISTCMP_BHATTACHARYYA);

    // 输出结果
    std::cout << "直方图比较结果: " << std::endl;
    std::cout << "相关性 (CORREL): " << correlate << std::endl;
    std::cout << "卡方 (CHISQR): " << chisquare << std::endl;
    std::cout << "相交 (INTERSECT): " << intersect << std::endl;
    std::cout << "巴氏距离 (BHATTACHARYYA): " << bhattacharyya << std::endl;

    // 解释结果
    std::cout << "\n结果解释: " << std::endl;
    std::cout << "- 相关性接近1表示图像相似" << std::endl;
    std::cout << "- 卡方接近0表示图像相似" << std::endl;
    std::cout << "- 相交接近1表示图像相似" << std::endl;
    std::cout << "- 巴氏距离接近0表示图像相似" << std::endl;

    return 0;
}
```

6.2 Python 示例

Listing 2: OpenCV 直方图比较 Python 示例

```python
import cv2
import numpy as np

def compare_histograms(img1_path, img2_path):
    # 读取图像
    img1 = cv2.imread(img1_path)
    img2 = cv2.imread(img2_path)

    if img1 is None or img2 is None:
        print("无法读取图像文件")
        return

    # 转换为HSV颜色空间
    hsv1 = cv2.cvtColor(img1, cv2.COLOR_BGR2HSV)
    hsv2 = cv2.cvtColor(img2, cv2.COLOR_BGR2HSV)

    # 计算2D直方图 (H和S通道)
    hist1 = cv2.calcHist([hsv1], [0, 1], None, [50, 60], [0, 180, 0, 256])
    hist2 = cv2.calcHist([hsv2], [0, 1], None, [50, 60], [0, 180, 0, 256])

    # 归一化直方图
    hist1 = cv2.normalize(hist1, hist1, 0, 1, cv2.NORM_MINMAX)
    hist2 = cv2.normalize(hist2, hist2, 0, 1, cv2.NORM_MINMAX)

    # 各种比较方法
    methods = {
        'CORREL': cv2.HISTCMP_CORREL,
        'CHISQR': cv2.HISTCMP_CHISQR,
        'INTERSECT': cv2.HISTCMP_INTERSECT,
        'BHATTACHARYYA': cv2.HISTCMP_BHATTACHARYYA,
        'HELLINGER': cv2.HISTCMP_HELLINGER
    }

    results = {}
    for method_name, method in methods.items():
        similarity = cv2.compareHist(hist1, hist2, method)
        results[method_name] = similarity

    return results

def print_comparison_results(results):
    print("直方图比较结果: ")
    print("=" * 40)

```

```python
    for method, value in results.items():
        print(f"{method:15} : {value:.6f}")

    print("\n结果解释指南: ")
    print("- CORREL: 接近1.0表示高度相似, 接近-1.0表示高度相反")
    print("- CHISQR: 接近0.0表示高度相似, 值越大差异越大")
    print("- INTERSECT: 接近1.0表示高度相似, 接近0.0表示不相似")
    print("- BHATTACHARYYA: 接近0.0表示高度相似, 接近1.0表示不相似")
    print("- HELLINGER: 接近0.0表示高度相似, 接近1.0表示不相似")

# 使用示例
if __name__ == "__main__":
    # 比较两幅图像
    results = compare_histograms("image1.jpg", "image2.jpg")

    if results:
        print_comparison_results(results)

        # 简单判断
        if results['CORREL'] > 0.8:
            print("\n判断: 两幅图像非常相似")
        elif results['CORREL'] > 0.5:
            print("\n判断: 两幅图像有一定相似性")
        else:
            print("\n判断: 两幅图像差异较大")

# 批量比较示例
def batch_compare(query_img_path, database_img_paths):
    """将查询图像与数据库中的多幅图像比较"""
    query_img = cv2.imread(query_img_path)
    query_hsv = cv2.cvtColor(query_img, cv2.COLOR_BGR2HSV)
    query_hist = cv2.calcHist([query_hsv], [0, 1], None, [50, 60], [0, 180, 0, 256])
    query_hist = cv2.normalize(query_hist, query_hist, 0, 1, cv2.NORM_MINMAX)

    similarities = []

    for img_path in database_img_paths:
        img = cv2.imread(img_path)
        if img is None:
            continue

        img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
        img_hist = cv2.calcHist([img_hsv], [0, 1], None, [50, 60], [0, 180, 0, 256])
        img_hist = cv2.normalize(img_hist, img_hist, 0, 1, cv2.NORM_MINMAX)

        # 使用相关性作为相似度指标
        similarity = cv2.compareHist(query_hist, img_hist, cv2.HISTCMP_CORREL)
        similarities.append((img_path, similarity))
```

```python

    # 按相似度排序
    similarities.sort(key=lambda x: x[1], reverse=True)

    return similarities
```

7 高级技巧与注意事项

7.1 直方图维度选择

- **1D 直方图**：适用于灰度图像或单个颜色通道，计算简单快速
- **2D 直方图**：两个颜色通道的组合（如 HSV 中的 H 和 S 通道），能更好地表示颜色分布
- **3D 直方图**：三个颜色通道的组合（如 RGB），计算量较大但信息更完整

7.2 直方图分箱策略

- **均匀分箱**：简单直观，但可能丢失细节信息
- **非均匀分箱**：根据数据分布调整 bin 边界，更精确但实现复杂
- **分箱数量**：过多会导致直方图稀疏，过少会丢失信息。通常选择 50-256 个 bin

7.3 颜色空间选择

- **RGB**：直观但受光照影响大，不适合直接比较
- **HSV/HSL**：将亮度（Value/Lightness）与颜色（Hue, Saturation）分离，更适合颜色比较
- **Lab**：感知均匀的颜色空间，与人眼感知更一致
- **YCrCb**：将亮度（Y）与色度（Cr, Cb）分离，常用于视频压缩

7.4 性能优化

- **降采样**：先缩小图像尺寸再计算直方图，大幅减少计算量
- **特征选择**：仅使用重要的颜色通道，减少直方图维度
- **缓存机制**：对静态图像预计算直方图，避免重复计算
- **并行计算**：多幅图像同时比较，利用多核处理器加速

8 数学公式补充

直方图比较涉及的主要数学公式总结：

8.1 归一化直方图

$$p(i) = \frac{h(i)}{\sum_{j=0}^{L-1} h(j)} = \frac{h(i)}{MN}$$

8.2 累积分布函数

$$C(k) = \sum_{i=0}^{k} p(i)$$

8.3 各种距离度量公式汇总

$$d_{\text{correl}}(H_1, H_2) = \frac{\sum_I (H_1(I) - \bar{H}_1)(H_2(I) - \bar{H}_2)}{\sqrt{\sum_I (H_1(I) - \bar{H}_1)^2 \sum_I (H_2(I) - \bar{H}_2)^2}} \tag{6}$$

$$d_{\text{chisqr}}(H_1, H_2) = \sum_I \frac{(H_1(I) - H_2(I))^2}{H_1(I) + H_2(I)} \tag{7}$$

$$d_{\text{intersect}}(H_1, H_2) = \sum_I \min(H_1(I), H_2(I)) \tag{8}$$

$$d_{\text{bhatt}}(H_1, H_2) = \sqrt{1 - \frac{1}{\sqrt{\bar{H}_1 \bar{H}_2 N^2}} \sum_I \sqrt{H_1(I) \cdot H_2(I)}} \tag{9}$$

$$d_{\text{hellinger}}(H_1, H_2) = \sqrt{1 - \sum_I \sqrt{\frac{H_1(I)}{N_1} \cdot \frac{H_2(I)}{N_2}}} \tag{10}$$

9 总结与展望

直方图比较是计算机视觉中基础而重要的技术，具有计算简单、实现容易、对几何变换有一定鲁棒性等优点。OpenCV 提供了完善的直方图计算和比较函数，方便开发者快速实现图像相似性分析。

在实际应用中，需要根据具体任务选择合适的直方图维度、分箱策略、颜色空间和距离度量方法。对于彩色图像，推荐使用 HSV 颜色空间的 2D 直方图（H 和 S 通道）进行比较，并使用相关性或巴氏距离作为相似度指标。

未来发展方向包括：

- **深度学习结合**：将传统直方图特征与深度学习特征结合，提高相似度度量的准确性
- **自适应参数选择**：根据图像内容自动选择最优的直方图参数
- **实时性能优化**：利用 GPU 加速和近似算法，实现大规模图像的实时比较
- **多特征融合**：结合颜色、纹理、形状等多种特征，提高图像检索的精度

注记 9.1. *在实际应用中，直方图比较通常作为图像相似性分析的第一步，可以与其他特征（如 SIFT、SURF、深度学习特征）结合使用，以获得更好的效果。同时，需要注意直方图比较对颜色变化敏感，但对几何变换（旋转、缩放）相对鲁棒的特点。*

参考文献

[1] OpenCV Documentation. *Histogram Comparison*. https://docs.opencv.org/

[2] Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.

[3] Szeliski, R. (2010). *Computer Vision: Algorithms and Applications*. Springer.